Topic	Description
	that allow the shaders to traverse the vertices in a specified order.
Port the GLSL	Once you've moved over the code that creates and configures your buffers and shader objects, it's time to port the code inside those shaders from OpenGL ES 2.0's GL Shader Language (GLSL) to Direct3D 11's High-level Shader Language (HLSL).
Draw to the screen	Finally, we port the code that draws the spinning cube to the screen.

Additional resources

- Prepare your dev environment for UWP DirectX game development
- Create a new DirectX 11 project for UWP
- Map OpenGL ES 2.0 concepts and infrastructure to Direct3D 11

Feedback

Port the shader objects

Article • 10/20/2022

Important APIs

- ID3D11Device
- ID3D11DeviceContext

When porting the simple renderer from OpenGL ES 2.0, the first step is to set up the equivalent vertex and fragment shader objects in Direct3D 11, and to make sure that the main program can communicate with the shader objects after they are compiled.

> **Note**
>
> Have you created a new Direct3D project? If not, follow the instructions in Create a new DirectX 11 project for Universal Windows Platform (UWP). This walkthrough assumes that you have the created the DXGI and Direct3D resources for drawing to the screen, and which are provided in the template.

Much like OpenGL ES 2.0, the compiled shaders in Direct3D must be associated with a drawing context. However, Direct3D does not have the concept of a shader program object per se; instead, you must assign the shaders directly to an ID3D11DeviceContext. This step follows the OpenGL ES 2.0 process for creating and binding shader objects, and provides you with the corresponding API behaviors in Direct3D.

Instructions

Step 1: Compile the shaders

In this simple OpenGL ES 2.0 sample, the shaders are stored as text files and loaded as string data for run-time compilation.

OpenGL ES 2.0: Compile a shader

syntax

```
GLuint __cdecl CompileShader (GLenum shaderType, const char *shaderSrcStr)
// shaderType can be GL_VERTEX_SHADER or GL_FRAGMENT_SHADER. Returns 0 if
compilation fails.
{
  GLuint shaderHandle;
  GLint compiledShaderHandle;
```

```
  // Create an empty shader object.
  shaderHandle = glCreateShader(shaderType);

  if (shaderHandle == 0)
  return 0;

  // Load the GLSL shader source as a string value. You could obtain it from
  // from reading a text file or hardcoded.
  glShaderSource(shaderHandle, 1, &shaderSrcStr, NULL);

  // Compile the shader.
  glCompileShader(shaderHandle);

  // Check the compile status
  glGetShaderiv(shaderHandle, GL_COMPILE_STATUS, &compiledShaderHandle);

  if (!compiledShaderHandle) // error in compilation occurred
  {
    // Handle any errors here.

    glDeleteShader(shaderHandle);
    return 0;
  }

  return shaderHandle;

}
```

In Direct3D, shaders are not compiled during run-time; they are always compiled to CSO files when the rest of the program is compiled. When you compile your app with Microsoft Visual Studio, the HLSL files are compiled to CSO (.cso) files that your app must load. Make sure you include these CSO files with your app when you package it!

> **Note** The following example performs the shader loading and compilation asynchronously using the **auto** keyword and lambda syntax. ReadDataAsync() is a method implemented for the template that reads in a CSO file as an array of byte data (fileData).

Direct3D 11: Compile a shader

syntax

```
auto loadVSTask = DX::ReadDataAsync(m_projectDir +
"SimpleVertexShader.cso");
auto loadPSTask = DX::ReadDataAsync(m_projectDir + "SimplePixelShader.cso");

auto createVSTask = loadVSTask.then([this](Platform::Array<byte>^ fileData)
```

```
{

m_d3dDevice->CreateVertexShader(
  fileData->Data,
  fileData->Length,
  nullptr,
  &m_vertexShader);

auto createPSTask = loadPSTask.then([this](Platform::Array<byte>^ fileData)
{
  m_d3dDevice->CreatePixelShader(
    fileData->Data,
    fileData->Length,
    nullptr,
    &m_pixelShader;
};
```

Step 2: Create and load the vertex and fragment (pixel) shaders

OpenGL ES 2.0 has the notion of a shader "program", which serves as the interface between the main program running on the CPU and the shaders, which are executed on the GPU. Shaders are compiled (or loaded from compiled sources) and associated with a program, which enables execution on the GPU.

OpenGL ES 2.0: Loading the vertex and fragment shaders into a shading program

syntax

```
GLuint __cdecl LoadShaderProgram (const char *vertShaderSrcStr, const char
*fragShaderSrcStr)
{
  GLuint programObject, vertexShaderHandle, fragmentShaderHandle;
  GLint linkStatusCode;

  // Load the vertex shader and compile it to an internal executable format.
  vertexShaderHandle = CompileShader(GL_VERTEX_SHADER, vertShaderSrcStr);
  if (vertexShaderHandle == 0)
  {
    glDeleteShader(vertexShaderHandle);
    return 0;
  }

   // Load the fragment/pixel shader and compile it to an internal
executable format.
  fragmentShaderHandle = CompileShader(GL_FRAGMENT_SHADER,
fragShaderSrcStr);
  if (fragmentShaderHandle == 0)
  {
    glDeleteShader(fragmentShaderHandle);
```

```
    return 0;
  }

  // Create the program object proper.
  programObject = glCreateProgram();

  if (programObject == 0)    return 0;

  // Attach the compiled shaders
  glAttachShader(programObject, vertexShaderHandle);
  glAttachShader(programObject, fragmentShaderHandle);

  // Compile the shaders into binary executables in memory and link them to
the program object..
  glLinkProgram(programObject);

  // Check the project object link status and determine if the program is
available.
  glGetProgramiv(programObject, GL_LINK_STATUS, &linkStatusCode);

  if (!linkStatusCode) // if link status <> 0
  {
    // Linking failed; delete the program object and return a failure code
(0).

    glDeleteProgram (programObject);
    return 0;
  }

  // Deallocate the unused shader resources. The actual executables are part
of the program object.
  glDeleteShader(vertexShaderHandle);
  glDeleteShader(fragmentShaderHandle);

  return programObject;
}

// ...

glUseProgram(renderer->programObject);
```

Direct3D does not have the concept of a shader program object. Rather, the shaders are created when one of the shader creation methods on the **ID3D11Device** interface (such as **ID3D11Device::CreateVertexShader** or **ID3D11Device::CreatePixelShader**) is called. To set the shaders for the current drawing context, we provide them to corresponding **ID3D11DeviceContext** with a set shader method, such as **ID3D11DeviceContext::VSSetShader** for the vertex shader or **ID3D11DeviceContext::PSSetShader** for the fragment shader.

Direct3D 11: Set the shaders for the graphics device drawing context.

```
```

syntax

```
m_d3dContext->VSSetShader(
  m_vertexShader.Get(),
  nullptr,
  0);

m_d3dContext->PSSetShader(
  m_pixelShader.Get(),
  nullptr,
  0);
```

Step 3: Define the data to supply to the shaders

In our OpenGL ES 2.0 example, we have one **uniform** to declare for the shader pipeline:

- **u_mvpMatrix**: a 4x4 array of floats that represents the final model-view-projection transformation matrix that takes the model coordinates for the cube and transforms them into 2D projection coordinates for scan conversion.

And two **attribute** values for the vertex data:

- **a_position**: a 4-float vector for the model coordinates of a vertex.
- **a_color**: A 4-float vector for the RGBA color value associated with the vertex.

Open GL ES 2.0: GLSL definitions for the uniforms and attributes

syntax

```
uniform mat4 u_mvpMatrix;
attribute vec4 a_position;
attribute vec4 a_color;
```

The corresponding main program variables are defined as fields on the renderer object, in this case. (Refer to the header in How to: port a simple OpenGL ES 2.0 renderer to Direct3D 11.) Once we've done that, we need to specify the locations in memory where the main program will supply these values for the shader pipeline, which we typically do right before a draw call:

OpenGL ES 2.0: Marking the location of the uniform and attribute data

syntax

```
// Inform the shader of the attribute locations
loc = glGetAttribLocation(renderer->programObject, "a_position");
glVertexAttribPointer(loc, 3, GL_FLOAT, GL_FALSE,
```

```
    sizeof(Vertex), 0);
glEnableVertexAttribArray(loc);

loc = glGetAttribLocation(renderer->programObject, "a_color");
glVertexAttribPointer(loc, 4, GL_FLOAT, GL_FALSE,
    sizeof(Vertex), (GLvoid*) (sizeof(float) * 3));
glEnableVertexAttribArray(loc);


// Inform the shader program of the uniform location
renderer->mvpLoc = glGetUniformLocation(renderer->programObject,
"u_mvpMatrix");
```

Direct3D does not have the concept of an "attribute" or a "uniform" in the same sense (or, at least, it does not share this syntax). Rather, it has constant buffers, represented as Direct3D subresources -- resources that are shared between the main program and the shader programs. Some of these subresources, such as vertex positions and colors, are described as HLSL semantics. For more info on constant buffers and HLSL semantics as they relate to OpenGL ES 2.0 concepts, read Port frame buffer objects, uniforms, and attributes.

When moving this process to Direct3D, we convert the uniform to a Direct3D constant buffer (cbuffer) and assign it a register for lookup with the **register** HLSL semantic. The two vertex attributes are handled as input elements to the shader pipeline stages, and are also assigned HLSL semantics (POSITION and COLOR0) that inform the shaders. The pixel shader takes an SV_POSITION, with the SV_ prefix indicating that it is a system value generated by the GPU. (In this case, it is a pixel position generated during scan conversion.) VertexShaderInput and PixelShaderInput are not declared as constant buffers because the former will be used to define the vertex buffer (see Port the vertex buffers and data), and the data for the latter is generated as the result of a previous stage in the pipeline, which in this case is the vertex shader.

Direct3D: HLSL definitions for the constant buffers and vertex data

syntax

```
cbuffer ModelViewProjectionConstantBuffer : register(b0)
{
  matrix mvp;
};

// Per-vertex data used as input to the vertex shader.
struct VertexShaderInput
{
  float4 pos : POSITION;
  float4 color : COLOR0;
};
```

```
// Per-vertex color data passed through the pixel shader.
struct PixelShaderInput
{
  float4 pos : SV_POSITION;
  float3 color : COLOR0;
};
```

For more info on porting to constant buffers and the application of HLSL semantics, read Port frame buffer objects, uniforms, and attributes.

Here are the structures for the layout of the data passed to the shader pipeline with a constant or vertex buffer.

Direct3D 11: Declaring the constant and vertex buffers layout

syntax

```
// Constant buffer used to send MVP matrices to the vertex shader.
struct ModelViewProjectionConstantBuffer
{
  DirectX::XMFLOAT4X4 modelViewProjection;
};

// Used to send per-vertex data to the vertex shader.
struct VertexPositionColor
{
  DirectX::XMFLOAT4 pos;
  DirectX::XMFLOAT4 color;
};
```

Use the DirectXMath XM* types for your constant buffer elements, since they provide proper packing and alignment for the contents when they are sent to the shader pipeline. If you use standard Windows platform float types and arrays, you must perform the packing and alignment yourself.

To bind a constant buffer, create a layout description as a **CD3D11_BUFFER_DESC** structure, and pass it to **ID3DDevice::CreateBuffer**. Then, in your render method, pass the constant buffer to **ID3D11DeviceContext::UpdateSubresource** before drawing.

Direct3D 11: Bind the constant buffer

syntax

```
CD3D11_BUFFER_DESC
constantBufferDesc(sizeof(ModelViewProjectionConstantBuffer),
D3D11_BIND_CONSTANT_BUFFER);

m_d3dDevice->CreateBuffer(
```

```
  &constantBufferDesc,
  nullptr,
  &m_constantBuffer);

// ...

// Only update shader resources that have changed since the last frame.
m_d3dContext->UpdateSubresource(
  m_constantBuffer.Get(),
  0,
  NULL,
  &m_constantBufferData,
  0,
  0);
```

The vertex buffer is created and updated similarly, and is discussed in the next step, Port the vertex buffers and data.

Next step

Port the vertex buffers and data

Related topics

How to: port a simple OpenGL ES 2.0 renderer to Direct3D 11

Port the vertex buffers and data

Port the GLSL

Draw to the screen

Feedback

Port the vertex buffers and data

Article • 10/20/2022

Important APIs

- ID3DDevice::CreateBuffer
- ID3DDeviceContext::IASetVertexBuffers
- ID3D11DeviceContext::IASetIndexBuffer

In this step, you'll define the vertex buffers that will contain your meshes and the index buffers that allow the shaders to traverse the vertices in a specified order.

At this point, let's examine the hardcoded model for the cube mesh we are using. Both representations have the vertices organized as a triangle list (as opposed to a strip or other more efficient triangle layout). All vertices in both representations also have associated indices and color values. Much of the Direct3D code in this topic refers to variables and objects defined in the Direct3D project.

Here's the cube for processing by OpenGL ES 2.0. In the sample implementation, each vertex is 7 float values: 3 position coordinates followed by 4 RGBA color values.

C++

```
#define CUBE_INDICES 36
#define CUBE_VERTICES 8

GLfloat cubeVertsAndColors[] =
{
  -0.5f, -0.5f,  0.5f, 0.0f, 0.0f, 1.0f, 1.0f,
  -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 1.0f,
  -0.5f,  0.5f,  0.5f, 0.0f, 1.0f, 1.0f, 1.0f,
  -0.5f,  0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f,
  0.5f, -0.5f,  0.5f, 1.0f, 0.0f, 1.0f, 1.0f,
  0.5f, -0.5f, -0.5f, 1.0f, 0.0f, 0.0f, 1.0f,
  0.5f,  0.5f,  0.5f, 1.0f, 1.0f, 1.0f, 1.0f,
  0.5f,  0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 1.0f
};

GLuint cubeIndices[] =
{
  0, 1, 2, // -x
  1, 3, 2,

  4, 6, 5, // +x
  6, 7, 5,

  0, 5, 1, // -y
  5, 6, 1,
```

```
  2, 6, 3, // +y
  6, 7, 3,

  0, 4, 2, // +z
  4, 6, 2,

  1, 7, 3, // -z
  5, 7, 1
};
```

And here's the same cube for processing by Direct3D 11.

C++

```
VertexPositionColor cubeVerticesAndColors[] =
// struct format is position, color
{
  {XMFLOAT3(-0.5f, -0.5f, -0.5f), XMFLOAT3(0.0f, 0.0f, 0.0f)},
  {XMFLOAT3(-0.5f, -0.5f,  0.5f), XMFLOAT3(0.0f, 0.0f, 1.0f)},
  {XMFLOAT3(-0.5f,  0.5f, -0.5f), XMFLOAT3(0.0f, 1.0f, 0.0f)},
  {XMFLOAT3(-0.5f,  0.5f,  0.5f), XMFLOAT3(0.0f, 1.0f, 1.0f)},
  {XMFLOAT3( 0.5f, -0.5f, -0.5f), XMFLOAT3(1.0f, 0.0f, 0.0f)},
  {XMFLOAT3( 0.5f, -0.5f,  0.5f), XMFLOAT3(1.0f, 0.0f, 1.0f)},
  {XMFLOAT3( 0.5f,  0.5f, -0.5f), XMFLOAT3(1.0f, 1.0f, 0.0f)},
  {XMFLOAT3( 0.5f,  0.5f,  0.5f), XMFLOAT3(1.0f, 1.0f, 1.0f)},
};

unsigned short cubeIndices[] =
{
  0, 2, 1, // -x
  1, 2, 3,

  4, 5, 6, // +x
  5, 7, 6,

  0, 1, 5, // -y
  0, 5, 4,

  2, 6, 7, // +y
  2, 7, 3,

  0, 4, 6, // -z
  0, 6, 2,

  1, 3, 7, // +z
  1, 7, 5
};
```

Reviewing this code, you notice that the cube in the OpenGL ES 2.0 code is represented in a right-hand coordinate system, whereas the cube in the Direct3D-specific code is

represented in a left-hand coordinate system. When importing your own mesh data, you must reverse the z-axis coordinates for your model and change the indices for each mesh accordingly to traverse the triangles according to the change in the coordinate system.

Assuming that we have successfully moved the cube mesh from the right-handed OpenGL ES 2.0 coordinate system to the left-handed Direct3D one, let's see how to load the cube data for processing in both models.

Instructions

Step 1: Create an input layout

In OpenGL ES 2.0, your vertex data is supplied as attributes that will be supplied to and read by the shader objects. You typically provide a string that contains the attribute name used in the shader's GLSL to the shader program object, and get a memory location back that you can supply to the shader. In this example, a vertex buffer object contains a list of custom Vertex structures, defined and formatted as follows:

OpenGL ES 2.0: Configure the attributes that contain the per-vertex information.

syntax

```
typedef struct
{
  GLfloat pos[3];
  GLfloat rgba[4];
} Vertex;
```

In OpenGL ES 2.0, input layouts are implicit; you take a general purpose GL_ELEMENT_ARRAY_BUFFER and supply the stride and offset such that the vertex shader can interpret the data after uploading it. You inform the shader before rendering which attributes map to which portions of each block of vertex data with **glVertexAttribPointer**.

In Direct3D, you must provide an input layout to describe the structure of the vertex data in the vertex buffer when you create the buffer, instead of before you draw the geometry. To do this, you use an input layout which corresponds to layout of the data for our individual vertices in memory. It is very important to specify this accurately!

Here, you create an input description as an array of **D3D11_INPUT_ELEMENT_DESC** structures.

Direct3D: Define an input layout description.

syntax

```
struct VertexPositionColor
{
  DirectX::XMFLOAT3 pos;
  DirectX::XMFLOAT3 color;
};

// ...

const D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
{
  { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
  { "COLOR",    0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
};
```

This input description defines a vertex as a pair of 2 3-coordinate vectors: one 3D vector to store the position of the vertex in model coordinates, and another 3D vector to store the RGB color value associated with the vertex. In this case, you use 3x32 bit floating point format, elements of which we represent in code as `XMFLOAT3(X.Xf, X.Xf, X.Xf)`. You should use types from the DirectXMath library whenever you are handling data that will be used by a shader, as it ensure the proper packing and alignment of that data. (For example, use **XMFLOAT3** or **XMFLOAT4** for vector data, and **XMFLOAT4X4** for matrices.)

For a list of all the possible format types, refer to **DXGI_FORMAT**.

With the per-vertex input layout defined, you create the layout object. In the following code, you write it to **m_inputLayout**, a variable of type **ComPtr** (which points to an object of type **ID3D11InputLayout**). **fileData** contains the compiled vertex shader object from the previous step, Port the shaders.

Direct3D: Create the input layout used by the vertex buffer.

syntax

```
Microsoft::WRL::ComPtr<ID3D11InputLayout>      m_inputLayout;

// ...

m_d3dDevice->CreateInputLayout(
  vertexDesc,
  ARRAYSIZE(vertexDesc),
```

```
  fileData->Data,
  fileShaderData->Length,
  &m_inputLayout
);
```

We've defined the input layout. Now, let's create a buffer that uses this layout and load it with the cube mesh data.

Step 2: Create and load the vertex buffer(s)

In OpenGL ES 2.0, you create a pair of buffers, one for the position data and one for the color data. (You could also create a struct that contains both and a single buffer.) You bind each buffer and write position and color data into them. Later, during your render function, bind the buffers again and provide the shader with the format of the data in the buffer so it can correctly interpret it.

OpenGL ES 2.0: Bind the vertex buffers

syntax

```
// upload the data for the vertex position buffer
glGenBuffers(1, &renderer->vertexBuffer);
glBindBuffer(GL_ARRAY_BUFFER, renderer->vertexBuffer);
glBufferData(GL_ARRAY_BUFFER, sizeof(VERTEX) * CUBE_VERTICES, renderer-
>vertices, GL_STATIC_DRAW);
```

In Direct3D, shader-accessible buffers are represented as **D3D11_SUBRESOURCE_DATA** structures. To bind the location of this buffer to shader object, you need to create a CD3D11_BUFFER_DESC structure for each buffer with **ID3DDevice::CreateBuffer**, and then set the buffer of the Direct3D device context by calling a set method specific to the buffer type, such as **ID3DDeviceContext::IASetVertexBuffers**.

When you set the buffer, you must set the stride (the size of the data element for an individual vertex) as well the offset (where the vertex data array actually starts) from the beginning of the buffer.

Notice that we assign the pointer to the **vertexIndices** array to the **pSysMem** field of the **D3D11_SUBRESOURCE_DATA** structure. If this isn't correct, your mesh will be corrupt or empty!

Direct3D: Create and set the vertex buffer

syntax

```
D3D11_SUBRESOURCE_DATA vertexBufferData = {0};
vertexBufferData.pSysMem = cubeVertices;
vertexBufferData.SysMemPitch = 0;
vertexBufferData.SysMemSlicePitch = 0;
CD3D11_BUFFER_DESC vertexBufferDesc(sizeof(cubeVertices),
D3D11_BIND_VERTEX_BUFFER);

m_d3dDevice->CreateBuffer(
  &vertexBufferDesc,
  &vertexBufferData,
  &m_vertexBuffer);

// ...

UINT stride = sizeof(VertexPositionColor);
UINT offset = 0;
m_d3dContext->IASetVertexBuffers(
  0,
  1,
  m_vertexBuffer.GetAddressOf(),
  &stride,
  &offset);
```

Step 3: Create and load the index buffer

Index buffers are an efficient way to allow the vertex shader to look up individual vertices. Although they are not required, we use them in this sample renderer. As with vertex buffers in OpenGL ES 2.0, an index buffer is created and bound as a general purpose buffer and the vertex indices you created earlier are copied into it.

When you're ready to draw, you bind both the vertex and the index buffer again, and call **glDrawElements**.

OpenGL ES 2.0: Send the index order to the draw call.

syntax

```
GLuint indexBuffer;

// ...

glGenBuffers(1, &renderer->indexBuffer);
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, renderer->indexBuffer);
glBufferData(GL_ELEMENT_ARRAY_BUFFER,
  sizeof(GLuint) * CUBE_INDICES,
  renderer->vertexIndices,
  GL_STATIC_DRAW);

// ...
```

```
// Drawing function

// Bind the index buffer
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, renderer->indexBuffer);
glDrawElements (GL_TRIANGLES, renderer->numIndices, GL_UNSIGNED_INT, 0);
```

With Direct3D, it's a bit very similar process, albeit a bit more didactic. Supply the index buffer as a Direct3D subresource to the **ID3D11DeviceContext** you created when you configured Direct3D. You do this by calling **ID3D11DeviceContext::IASetIndexBuffer** with the configured subresource for the index array, as follows. (Again, notice that you assign the pointer to the **cubeIndices** array to the **pSysMem** field of the **D3D11_SUBRESOURCE_DATA** structure.)

Direct3D: Create the index buffer.

syntax

```
m_indexCount = ARRAYSIZE(cubeIndices);

D3D11_SUBRESOURCE_DATA indexBufferData = {0};
indexBufferData.pSysMem = cubeIndices;
indexBufferData.SysMemPitch = 0;
indexBufferData.SysMemSlicePitch = 0;
CD3D11_BUFFER_DESC indexBufferDesc(sizeof(cubeIndices),
D3D11_BIND_INDEX_BUFFER);

m_d3dDevice->CreateBuffer(
  &indexBufferDesc,
  &indexBufferData,
  &m_indexBuffer);

// ...

m_d3dContext->IASetIndexBuffer(
  m_indexBuffer.Get(),
  DXGI_FORMAT_R16_UINT,
  0);
```

Later, you will draw the triangles with a call to **ID3D11DeviceContext::DrawIndexed** (or **ID3D11DeviceContext::Draw** for unindexed vertices), as follows. (For more details, jump ahead to Draw to the screen.)

Direct3D: Draw the indexed vertices.

syntax

```
m_d3dContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
m_d3dContext->IASetInputLayout(m_inputLayout.Get());
```

```
// ...

m_d3dContext->DrawIndexed(
  m_indexCount,
  0,
  0);
```

Previous step

Port the shader objects

Next step

Port the GLSL

Remarks

When structuring your Direct3D, separate the code that calls methods on **ID3D11Device** into a method that is called whenever the device resources need to be recreated. (In the Direct3D project template, this code is in the renderer object's **CreateDeviceResource** methods. The code that updates the device context (**ID3D11DeviceContext**), on the other hand, is placed in the **Render** method, since this is where you actually construct the shader stages and bind the data.

Related topics

- How to: port a simple OpenGL ES 2.0 renderer to Direct3D 11
- Port the shader objects
- Port the vertex buffers and data
- Port the GLSL

Port the GLSL

Article • 10/20/2022

Important APIs

- HLSL Semantics
- Shader Constants (HLSL)

Once you've moved over the code that creates and configures your buffers and shader objects, it's time to port the code inside those shaders from OpenGL ES 2.0's GL Shader Language (GLSL) to Direct3D 11's High-level Shader Language (HLSL).

In OpenGL ES 2.0, shaders return data after execution using intrinsics such as **gl_Position**, **gl_FragColor**, or **gl_FragData[n]** (where n is the index for a specific render target). In Direct3D, there are no specific intrinsics, and the shaders return data as the return type of their respective main() functions.

Data that you want interpolated between shader stages, such as the vertex position or normal, is handled through the use of the **varying** declaration. However, Direct3D doesn't have this declaration; rather, any data that you want passed between shader stages must be marked with an HLSL semantic. The specific semantic chosen indicates the purpose of the data, and is. For example, you'd declare vertex data that you want interpolated between the fragment shader as:

```
float4 vertPos : POSITION;
```

or

```
float4 vertColor : COLOR;
```

Where POSITION is the semantic used to indicate vertex position data. POSITION is also a special case, since after interpolation, it cannot be accessed by the pixel shader. Therefore, you must specify input to the pixel shader with SV_POSITION and the interpolated vertex data will be placed in that variable.

```
float4 position : SV_POSITION;
```

Semantics can be declared on the body (main) methods of shaders. For pixel shaders, SV_TARGET[n], which indicates a render target, is required on the body method. (SV_TARGET without a numeric suffix defaults to render target index 0.)

Also note that vertex shaders are required to output the SV_POSITION system value semantic. This semantic resolves the vertex position data to coordinate values where x is

between -1 and 1, y is between -1 and 1, z is divided by the original homogeneous coordinate w value (z/w), and w is 1 divided by the original w value (1/w). Pixel shaders use the SV_POSITION system value semantic to retrieve the pixel location on the screen, where x is between 0 and the render target width and y is between 0 and the render target height (each offset by 0.5). Feature level 9_x pixel shaders cannot read from the SV_POSITION value.

Constant buffers must be declared with **cbuffer** and be associated with a specific starting register for lookup.

Direct3D 11: An HLSL constant buffer declaration

syntax

```
cbuffer ModelViewProjectionConstantBuffer : register(b0)
{
  matrix mvp;
};
```

Here, the constant buffer uses register b0 to hold the packed buffer. All registers are referred to in the form b#. For more information on the HLSL implementation of constant buffers, registers, and data packing, read Shader Constants (HLSL).

Instructions

Step 1: Port the vertex shader

In our simple OpenGL ES 2.0 example, the vertex shader has three inputs: a constant model-view-projection 4x4 matrix, and two 4-coordinate vectors. These two vectors contain the vertex position and its color. The shader transforms the position vector to perspective coordinates and assigns it to the gl_Position intrinsic for rasterization. The vertex color is copied to a varying variable for interpolation during rasterization, as well.

OpenGL ES 2.0: Vertex shader for the cube object (GLSL)

syntax

```
uniform mat4 u_mvpMatrix;
attribute vec4 a_position;
attribute vec4 a_color;
varying vec4 destColor;

void main()
{
  gl_Position = u_mvpMatrix * a_position;
```

```
  destColor = a_color;
}
```

Now, in Direct3D, the constant model-view-projection matrix is contained in a constant buffer packed at register b0, and the vertex position and color are specifically marked with the appropriate respective HLSL semantics: POSITION and COLOR. Since our input layout indicates a specific arrangement of these two vertex values, you create a struct to hold them and declare it as the type for the input parameter on the shader body function (main). (You could also specify them as two individual parameters, but that could get cumbersome.) You also specify an output type for this stage, which contains the interpolated position and color, and declare it as the return value for the body function of the vertex shader.

Direct3D 11: Vertex shader for the cube object (HLSL)

syntax

```
cbuffer ModelViewProjectionConstantBuffer : register(b0)
{
  matrix mvp;
};

// Per-vertex data used as input to the vertex shader.
struct VertexShaderInput
{
  float3 pos : POSITION;
  float3 color : COLOR;
};

// Per-vertex color data passed through the pixel shader.
struct PixelShaderInput
{
  float3 pos : SV_POSITION;
  float3 color : COLOR;
};

PixelShaderInput main(VertexShaderInput input)
{
  PixelShaderInput output;
  float4 pos = float4(input.pos, 1.0f); // add the w-coordinate

  pos = mul(mvp, projection);
  output.pos = pos;

  output.color = input.color;

  return output;
}
```

The output data type, PixelShaderInput, is populated during rasterization and provided to the fragment (pixel) shader.

Step 2: Port the fragment shader

Our example fragment shader in GLSL is extremely simple: provide the gl_FragColor intrinsic with the interpolated color value. OpenGL ES 2.0 will write it to the default render target.

OpenGL ES 2.0: Fragment shader for the cube object (GLSL)

syntax

```
varying vec4 destColor;

void main()
{
  gl_FragColor = destColor;
}
```

Direct3D is almost as simple. The only significant difference is that the body function of the pixel shader must return a value. Since the color is a 4-coordinate (RGBA) float value, you indicate float4 as the return type, and then specify the default render target as the SV_TARGET system value semantic.

Direct3D 11: Pixel shader for the cube object (HLSL)

syntax

```
struct PixelShaderInput
{
  float4 pos : SV_POSITION;
  float3 color : COLOR;
};


float4 main(PixelShaderInput input) : SV_TARGET
{
  return float4(input.color, 1.0f);
}
```

The color for the pixel at the position is written to the render target. Now, let's see how to display the contents of that render target in Draw to the screen!

Previous step

Port the vertex buffers and data

Next step

Draw to the screen

Remarks

Understanding HLSL semantics and the packing of constant buffers can save you a bit of a debugging headache, as well as provide optimization opportunities. If you get a chance, read through Variable Syntax (HLSL), Introduction to Buffers in Direct3D 11, and How to: Create a Constant Buffer. If not, though, here's a few starting tips to keep in mind about semantics and constant buffers:

- Always double check your renderer's Direct3D configuration code to make sure that the structures for your constant buffers match the cbuffer struct declarations in your HLSL, and that the component scalar types match across both declarations.
- In your renderer's C++ code, use DirectXMath types in your constant buffer declarations to ensure proper data packing.
- The best way to efficiently use constant buffers is to organize shader variables into constant buffers based on their frequency of update. For example, if you have some uniform data that is updated once per frame, and other uniform data that is updated only when the camera moves, consider separating that data into two separate constant buffers.
- Semantics that you have forgotten to apply or which you have applied incorrectly will be your earliest source of shader compilation (FXC) errors. Double-check them! The docs can be a bit confusing, as many older pages and samples refer to different versions of HLSL semantics prior to Direct3D 11.
- Make sure you know which Direct3D feature level you are targeting for each shader. The semantics for feature level 9_* are different from those for 11_1.
- The SV_POSITION semantic resolves the associated post-interpolation position data to coordinate values where x is between 0 and the render target width, y is between 0 and the render target height, z is divided by the original homogeneous coordinate w value (z/w), and w is 1 divided by the original w value (1/w).

Related topics

How to: port a simple OpenGL ES 2.0 renderer to Direct3D 11

Port the shader objects

Port the vertex buffers and data

Draw to the screen

Draw to the screen

Article • 10/20/2022

Important APIs

- ID3D11Texture2D
- ID3D11RenderTargetView
- IDXGISwapChain1

Finally, we port the code that draws the spinning cube to the screen.

In OpenGL ES 2.0, your drawing context is defined as an EGLContext type, which contains the window and surface parameters as well the resources necessary for drawing to the render targets that will be used to compose the final image displayed to the window. You use this context to configure the graphics resources to correctly display the results of your shader pipeline on the display. One of the primary resources is the "back buffer" (or "frame buffer object") that contains the final, composited render targets, ready for presentation to the display.

With Direct3D, the process of configuring the graphics resources for drawing to the display is more didactic, and requires quite a few more APIs. (A Microsoft Visual Studio Direct3D template can significantly simplify this process, though!) To obtain a context (called a Direct3D device context), you must first obtain an ID3D11Device1 object, and use it to create and configure an ID3D11DeviceContext1 object. These two objects are used in conjunction to configure the specific resources you need for drawing to the display.

In short, the DXGI APIs contain primarily APIs for managing resources that directly pertain to the graphics adapter, and Direct3D contains the APIs that allow you to interface between the GPU and your main program running on the CPU.

For the purposes of comparison in this sample, here are the relevant types from each API:

- ID3D11Device1: provides a virtual representation of the graphics device and its resources.
- ID3D11DeviceContext1: provides the interface to configure buffers and issue rendering commands.
- IDXGISwapChain1: the swap chain is analogous to the back buffer in OpenGL ES 2.0. It is the region of memory on the graphics adapter that contains the final rendered image(s) for display. It is called the "swap chain" because it has several

buffers that can be written to and "swapped" to present the latest render to the screen.

- ID3D11RenderTargetView: this contains the 2D bitmap buffer that the Direct3D device context draws into, and which is presented by the swap chain. As with OpenGL ES 2.0, you can have multiple render targets, some of which are not bound to the swap chain but are used for multi-pass shading techniques.

In the template, the renderer object contains the following fields:

Direct3D 11: Device and device context declarations

syntax

```
Platform::Agile<Windows::UI::Core::CoreWindow>       m_window;

Microsoft::WRL::ComPtr<ID3D11Device1>                m_d3dDevice;
Microsoft::WRL::ComPtr<ID3D11DeviceContext1>          m_d3dContext;
Microsoft::WRL::ComPtr<IDXGISwapChain1>
m_swapChainCoreWindow;
Microsoft::WRL::ComPtr<ID3D11RenderTargetView>
m_d3dRenderTargetViewWin;
```

Here's how the back buffer is configured as a render target and provided to the swap chain.

syntax

```
ComPtr<ID3D11Texture2D> backBuffer;
m_swapChainCoreWindow->GetBuffer(0, IID_PPV_ARGS(backBuffer));
m_d3dDevice->CreateRenderTargetView(
  backBuffer.Get(),
  nullptr,
  &m_d3dRenderTargetViewWin);
```

The Direct3D runtime implicitly creates an IDXGISurface1 for the ID3D11Texture2D, which represents the texture as a "back buffer" that the swap chain can use for display.

The initialization and configuration of the Direct3D device and device context, as well as the render targets, can be found in the custom **CreateDeviceResources** and **CreateWindowSizeDependentResources** methods in the Direct3D template.

For more info on Direct3D device context as it relates to EGL and the EGLContext type, read Port EGL code to DXGI and Direct3D.

Instructions

Step 1: Rendering the scene and displaying it

After updating the cube data (in this case, by rotating it slightly around the y axis), the Render method sets the viewport to the dimensions of he drawing context (an EGLContext). This context contains the color buffer that will be displayed to the window surface (an EGLSurface), using the configured display (EGLDisplay). At this time, the example updates the vertex data attributes, re-binds the index buffer, draws the cube, and swaps in color buffer drawn by the shading pipeline to the display surface.

OpenGL ES 2.0: Rendering a frame for display

syntax

```
void Render(GraphicsContext *drawContext)
{
  Renderer *renderer = drawContext->renderer;

  int loc;

  // Set the viewport
  glViewport ( 0, 0, drawContext->width, drawContext->height );


  // Clear the color buffer
  glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
  glEnable(GL_DEPTH_TEST);


  // Use the program object
  glUseProgram (renderer->programObject);

  // Load the a_position attribute with the vertex position portion of a
vertex buffer element
  loc = glGetAttribLocation(renderer->programObject, "a_position");
  glVertexAttribPointer(loc, 3, GL_FLOAT, GL_FALSE,
      sizeof(Vertex), 0);
  glEnableVertexAttribArray(loc);

  // Load the a_color attribute with the color position portion of a vertex
buffer element
  loc = glGetAttribLocation(renderer->programObject, "a_color");
  glVertexAttribPointer(loc, 4, GL_FLOAT, GL_FALSE,
      sizeof(Vertex), (GLvoid*) (sizeof(float) * 3));
  glEnableVertexAttribArray(loc);

  // Bind the index buffer
  glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, renderer->indexBuffer);

  // Load the MVP matrix
  glUniformMatrix4fv(renderer->mvpLoc, 1, GL_FALSE, (GLfloat*) &renderer-
>mvpMatrix.m[0][0]);
```

```
  // Draw the cube
  glDrawElements(GL_TRIANGLES, renderer->numIndices, GL_UNSIGNED_INT, 0);

  eglSwapBuffers(drawContext->eglDisplay, drawContext->eglSurface);
}
```

In Direct3D 11, the process is very similar. (We're assuming that you're using the viewport and render target configuration from the Direct3D template.

- Update the constant buffers (the model-view-projection matrix, in this case) with calls to **ID3D11DeviceContext1::UpdateSubresource**.
- Set the vertex buffer with **ID3D11DeviceContext1::IASetVertexBuffers**.
- Set the index buffer with **ID3D11DeviceContext1::IASetIndexBuffer**.
- Set the specific triangle topology (a triangle list) with **ID3D11DeviceContext1::IASetPrimitiveTopology**.
- Set the input layout of the vertex buffer with **ID3D11DeviceContext1::IASetInputLayout**.
- Bind the vertex shader with **ID3D11DeviceContext1::VSSetShader**.
- Bind the fragment shader with **ID3D11DeviceContext1::PSSetShader**.
- Send the indexed vertices through the shaders and output the color results to the render target buffer with **ID3D11DeviceContext1::DrawIndexed**.
- Display the render target buffer with **IDXGISwapChain1::Present1**.

Direct3D 11: Rendering a frame for display

syntax

```
void RenderObject::Render()
{
  // ...

  // Only update shader resources that have changed since the last frame.
  m_d3dContext->UpdateSubresource(
    m_constantBuffer.Get(),
    0,
    NULL,
    &m_constantBufferData,
    0,
    0);

  // Set up the IA stage corresponding to the current draw operation.
  UINT stride = sizeof(VertexPositionColor);
  UINT offset = 0;
  m_d3dContext->IASetVertexBuffers(
    0,
    1,
    m_vertexBuffer.GetAddressOf(),
```

```
    &stride,
    &offset);

  m_d3dContext->IASetIndexBuffer(
    m_indexBuffer.Get(),
    DXGI_FORMAT_R16_UINT,
    0);

  m_d3dContext-
>IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
  m_d3dContext->IASetInputLayout(m_inputLayout.Get());

  // Set up the vertex shader corresponding to the current draw operation.
  m_d3dContext->VSSetShader(
    m_vertexShader.Get(),
    nullptr,
    0);

  m_d3dContext->VSSetConstantBuffers(
    0,
    1,
    m_constantBuffer.GetAddressOf());

  // Set up the pixel shader corresponding to the current draw operation.
  m_d3dContext->PSSetShader(
    m_pixelShader.Get(),
    nullptr,
    0);

  m_d3dContext->DrawIndexed(
    m_indexCount,
    0,
    0);

    // ...

  m_swapChainCoreWindow->Present1(1, 0, &parameters);
}
```

Once **IDXGISwapChain1::Present1** is called, your frame is output to the configured display.

Previous step

Port the GLSL

Remarks

This example glosses over much of the complexity that goes into configuring device resources, especially for Universal Windows Platform (UWP) DirectX apps. We suggest you review the full template code, especially the parts that perform the window and device resource setup and management. UWP apps have to support rotation events as well as suspend/resume events, and the template demonstrates best practices for handling the loss of an interface or a change in the display parameters.

Related topics

- How to: port a simple OpenGL ES 2.0 renderer to Direct3D 11
- Port the shader objects
- Port the GLSL
- Draw to the screen

OpenGL ES 2.0 to Direct3D 11 reference

Article • 10/20/2022

Use these reference topics to look up API mapping and short code samples when porting from OpenGL ES 2.0 to Direct3D 11.

Topic	Description
GLSL-to-HLSL reference	You port your OpenGL Shader Language (GLSL) code to Microsoft High Level Shader Language (HLSL) code when you port your graphics architecture from OpenGL ES 2.0 to Direct3D 11 to create a game for Universal Windows Platform (UWP).

GLSL-to-HLSL reference

Article • 10/20/2022

You port your OpenGL Shader Language (GLSL) code to Microsoft High Level Shader Language (HLSL) code when you port your graphics architecture from OpenGL ES 2.0 to Direct3D 11 to create a game for Universal Windows Platform (UWP). The GLSL that is referred to herein is compatible with OpenGL ES 2.0; the HLSL is compatible with Direct3D 11. For info about the differences between Direct3D 11 and previous versions of Direct3D, see Feature mapping.

Comparing OpenGL ES 2.0 with Direct3D 11

OpenGL ES 2.0 and Direct3D 11 have many similarities. They both have similar rendering pipelines and graphics features. But Direct3D 11 is a rendering implementation and API, not a specification; OpenGL ES 2.0 is a rendering specification and API, not an implementation. Direct3D 11 and OpenGL ES 2.0 generally differ in these ways:

Expand table

OpenGL ES 2.0	Direct3D 11
Hardware and operating system agnostic specification with vendor provided implementations	Microsoft implementation of hardware abstraction and certification on Windows platforms
Abstracted for hardware diversity, runtime manages most resources	Direct access to hardware layout; app can manage resources and processing
Provides higher-level modules via third-party libraries (for example, Simple DirectMedia Layer (SDL))	Higher-level modules, like Direct2D, are built upon lower modules to simplify development for Windows apps

OpenGL ES 2.0	Direct3D 11
Hardware vendors differentiate via extensions	Microsoft adds optional features to the API in a generic way so they aren't specific to any particular hardware vendor

GLSL and HLSL generally differ in these ways:

Expand table

GLSL	HLSL
Procedural, step-centric (C like)	Object oriented, data-centric (C++ like)
Shader compilation integrated into the graphics API	The HLSL compiler compiles the shader to an intermediate binary representation before Direct3D passes it to the driver. **Note** This binary representation is hardware independent. It's typically compiled at app build time, rather than at app run time.
Variable storage modifiers	Constant buffers and data transfers via input layout declarations
Types Typical vector type: vec2/3/4 lowp, mediump, highp	Typical vector type: float2/3/4 min10float, min16float
texture2D [Function]	texture.Sample [datatype.Function]
sampler2D [datatype]	Texture2D [datatype]
Row-major matrices (default)	Column-major matrices (default) **Note** Use the **row_major** type-modifier to change the layout for one variable. For more info, see **Variable Syntax**. You can also specify

GLSL	HLSL
	a compiler flag or a pragma to change the global default.
Fragment shader	Pixel shader

> **Note** HLSL has textures and samplers as two separate objects. In GLSL, like Direct3D 9, the texture binding is part of the sampler state.

In GLSL, you present much of the OpenGL state as pre-defined global variables. For example, with GLSL, you use the **gl_Position** variable to specify vertex position and the **gl_FragColor** variable to specify fragment color. In HLSL, you pass Direct3D state explicitly from the app code to the shader. For example, with Direct3D and HLSL, the input to the vertex shader must match the data format in the vertex buffer, and the structure of a constant buffer in the app code must match the structure of a constant buffer (cbuffer) in shader code.

Porting GLSL variables to HLSL

In GLSL, you apply modifiers (qualifiers) to a global shader variable declaration to give that variable a specific behavior in your shaders. In HLSL, you don't need these modifiers because you define the flow of the shader with the arguments that you pass to your shader and that you return from your shader.

Expand table

GLSL variable behavior	HLSL equivalent
uniform You pass a uniform variable from the app code into either or both vertex and fragment shaders. You must set the values of all uniforms before you draw any triangles with those shaders so their values stay the same throughout the drawing of a triangle mesh. These values are uniform. Some uniforms are	Use constant buffer. See How to: Create a Constant Buffer and Shader Constants.

GLSL variable behavior	HLSL equivalent
set for the entire frame and others uniquely to one particular vertex-pixel shader pair. Uniform variables are per-polygon variables.	
varying You initialize a varying variable inside the vertex shader and pass it through to an identically named varying variable in the fragment shader. Because the vertex shader only sets the value of the varying variables at each vertex, the rasterizer interpolates those values (in a perspective-correct manner) to generate per fragment values to pass into the fragment shader. These variables vary across each triangle.	Use the structure that you return from your vertex shader as the input to your pixel shader. Make sure the semantic values match.
attribute An attribute is a part of the description of a vertex that you pass from the app code to the vertex shader alone. Unlike a uniform, you set each attribute's value for each vertex, which, in turn, allows each vertex to have a different value. Attribute variables are per-vertex variables.	Define a vertex buffer in your Direct3D app code and match it to the vertex input defined in the vertex shader. Optionally, define an index buffer. See How to: Create a Vertex Buffer and How to: Create an Index Buffer. Create an input layout in your Direct3D app code and match semantic values with those in the vertex input. See Create the input layout.
const Constants that are compiled into the shader and never change.	Use a **static const**. **static** means the value isn't exposed to constant buffers, **const** means the shader can't change the value. So, the value is known at compile time based on its initializer.

In GLSL, variables without modifiers are just ordinary global variables that are private to each shader.

When you pass data to textures (Texture2D in HLSL) and their associated samplers (SamplerState in HLSL), you typically declare them as global variables in the pixel shader.

Porting GLSL types to HLSL

Use this table to port your GLSL types to HLSL.

Expand table

GLSL type	HLSL type
scalar types: float, int, bool	scalar types: float, int, bool also, uint, double For more info, see Scalar Types.

vector type
 	
• floating-point vector: vec2, vec3, vec4
• Boolean vector: bvec2, bvec3, bvec4
• signed integer vector: ivec2, ivec3, ivec4
	vector type
	
	• float2, float3, float4, and float1
	• bool2, bool3, bool4, and bool1
	• int2, int3, int4, and int1
	• These types also have vector expansions similar to float, bool, and int:
	  ◦ uint
	  ◦ min10float, min16float
	  ◦ min12int, min16int
	  ◦ min16uint
	
	For more info, see Vector Type and Keywords.
	
	vector is also type defined as float4 (typedef vector <float, 4> vector;). For more info, see User-Defined Type.

matrix type
 	
• mat2: 2x2 float matrix
• mat3: 3x3 float matrix
• mat4: 4x4 float matrix
	matrix type
	
	• float2x2
	• float3x3
	• float4x4
	• also, float1x1, float1x2, float1x3, float1x4, float2x1, float2x3, float2x4, float3x1, float3x2, float3x4, float4x1, float4x2, float4x3
	• These types also have matrix expansions similar to float:
	  ◦ int, uint, bool
	  ◦ min10float, min16float
	  ◦ min12int, min16int
	  ◦ min16uint
	
	You can also use the matrix type to define a matrix.
	
	For example: matrix <float, 2, 2> fMatrix = {0.0f, 0.1, 2.1f, 2.2f};
	
	matrix is also type defined as float4x4 (typedef matrix <float, 4, 4> matrix;). For more info, see User-Defined Type.

GLSL type	HLSL type
precision qualifiers for float, int, sampler • highp This qualifier provides minimum precision requirements that are greater than that provided by min16float and less than a full 32-bit float. Equivalent in HLSL is: highp float -> float highp int -> int • mediump This qualifier applied to float and int is equivalent to min16float and min12int in HLSL. Minimum 10 bits of mantissa, not like min10float. • lowp This qualifier applied to float provides a floating point range of -2 to 2. Equivalent to min10float in HLSL.	precision types • min16float: minimum 16-bit floating point value • min10float Minimum fixed-point signed 2.8 bit value (2 bits of whole number and 8 bits fractional component). The 8-bit fractional component can be inclusive of 1 instead of exclusive to give it the full inclusive range of -2 to 2. • min16int: minimum 16-bit signed integer • min12int: minimum 12-bit signed integer This type is for 10Level9 (9_x feature levels) in which integers are represented by floating point numbers. This is the precision you can get when you emulate an integer with a 16-bit floating point number. • min16uint: minimum 16-bit unsigned integer For more info, see Scalar Types and Using HLSL minimum precision.
sampler2D	Texture2D
samplerCube	TextureCube

Porting GLSL pre-defined global variables to HLSL

Use this table to port GLSL pre-defined global variables to HLSL.

Expand table

GLSL pre-defined global variable	HLSL semantics
gl_Position This variable is type **vec4**.	SV_Position POSITION in Direct3D 9

GLSL pre-defined global variable	HLSL semantics
Vertex position for example - gl_Position = position;	This semantic is type **float4**. Vertex shader output Vertex position for example - float4 vPosition : SV_Position;
gl_PointSize This variable is type **float**. Point size	PSIZE No meaning unless you target Direct3D 9 This semantic is type **float**. Vertex shader output Point size
gl_FragColor This variable is type **vec4**. Fragment color for example - gl_FragColor = vec4(colorVarying, 1.0);	SV_Target COLOR in Direct3D 9 This semantic is type **float4**. Pixel shader output Pixel color for example - float4 Color[4] : SV_Target;
gl_FragData[n] This variable is type **vec4**. Fragment color for color attachment n	SV_Target[n] This semantic is type **float4**. Pixel shader output value that is stored in n render target, where 0 <= n <= 7.
gl_FragCoord This variable is type **vec4**. Fragment position within frame buffer	SV_Position Not available in Direct3D 9 This semantic is type **float4**. Pixel shader input Screen space coordinates for example - float4 screenSpace : SV_Position
gl_FrontFacing This variable is type **bool**.	SV_IsFrontFace VFACE in Direct3D 9 SV_IsFrontFace is type **bool**.

GLSL pre-defined global variable	HLSL semantics
Determines whether fragment belongs to a front-facing primitive.	VFACE is type **float**. Pixel shader input Primitive facing
gl_PointCoord This variable is type **vec2**. Fragment position within a point (point rasterization only)	SV_Position VPOS in Direct3D 9 SV_Position is type **float4**. VPOS is type **float2**. Pixel shader input The pixel or sample position in screen space for example - float4 pos : SV_Position
gl_FragDepth This variable is type **float**. Depth buffer data	SV_Depth DEPTH in Direct3D 9 SV_Depth is type **float**. Pixel shader output Depth buffer data

You use semantics to specify position, color, and so on for vertex shader input and pixel shader input. You must match the semantics values in the input layout with the vertex shader input. For examples, see Examples of porting GLSL variables to HLSL. For more info about the HLSL semantics, see Semantics.

Examples of porting GLSL variables to HLSL

Here we show examples of using GLSL variables in OpenGL/GLSL code and then the equivalent example in Direct3D/HLSL code.

Uniform, attribute, and varying in GLSL

OpenGL app code

```
syntax
```

```
// Uniform values can be set in app code and then processed in the shader
code.
uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

// Incoming position of vertex
attribute vec4 position;

// Incoming color for the vertex
attribute vec3 color;

// The varying variable tells the shader pipeline to pass it
// on to the fragment shader.
varying vec3 colorVarying;
```

GLSL vertex shader code

syntax

```
//The shader entry point is the main method.
void main()
{
colorVarying = color; //Use the varying variable to pass the color to the
fragment shader
gl_Position = position; //Copy the position to the gl_Position pre-defined
global variable
}
```

GLSL fragment shader code

syntax

```
void main()
{
//Pad the colorVarying vec3 with a 1.0 for alpha to create a vec4 color
//and assign that color to the gl_FragColor pre-defined global variable
//This color then becomes the fragment's color.
gl_FragColor = vec4(colorVarying, 1.0);
}
```

Constant buffers and data transfers in HLSL

Here is an example of how you pass data to the HLSL vertex shader that then flows through to the pixel shader. In your app code, define a vertex and a constant buffer. Then, in your vertex shader code, define the constant buffer as a cbuffer and store the

per-vertex data and the pixel shader input data. Here we use structures called **VertexShaderInput** and **PixelShaderInput**.

Direct3D app code

C++

```
struct ConstantBuffer
{
    XMFLOAT4X4 model;
    XMFLOAT4X4 view;
    XMFLOAT4X4 projection;
};
struct SimpleCubeVertex
{
    XMFLOAT3 pos;   // position
    XMFLOAT3 color; // color
};

 // Create an input layout that matches the layout defined in the vertex
shader code.
 const D3D11_INPUT_ELEMENT_DESC basicVertexLayoutDesc[] =
 {
     { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0,  0,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
     { "COLOR",    0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
 };

// Create vertex and index buffers that define a geometry.
```

HLSL vertex shader code

syntax

```
cbuffer ModelViewProjectionCB : register( b0 )
{
    matrix model;
    matrix view;
    matrix projection;
};
// The POSITION and COLOR semantics must match the semantics in the input
layout Direct3D app code.
struct VertexShaderInput
{
    float3 pos : POSITION; // Incoming position of vertex
    float3 color : COLOR; // Incoming color for the vertex
};

struct PixelShaderInput
{
    float4 pos : SV_Position; // Copy the vertex position.
```

```
    float4 color : COLOR; // Pass the color to the pixel shader.
};

PixelShaderInput main(VertexShaderInput input)
{
    PixelShaderInput vertexShaderOutput;

    // shader source code

    return vertexShaderOutput;
}
```

HLSL pixel shader code

syntax

```
// Collect input from the vertex shader.
// The COLOR semantic must match the semantic in the vertex shader code.
struct PixelShaderInput
{
    float4 pos : SV_Position;
    float4 color : COLOR; // Color for the pixel
};

// Set the pixel color value for the renter target.
float4 main(PixelShaderInput input) : SV_Target
{
    return input.color;
}
```

Examples of porting OpenGL rendering code to Direct3D

Here we show an example of rendering in OpenGL ES 2.0 code and then the equivalent example in Direct3D 11 code.

OpenGL rendering code

syntax

```
// Bind shaders to the pipeline.
// Both vertex shader and fragment shader are in a program.
glUseProgram(m_shader->getProgram());

// Input assembly
// Get the position and color attributes of the vertex.

m_positionLocation = glGetAttribLocation(m_shader->getProgram(),
"position");
```

```
glEnableVertexAttribArray(m_positionLocation);

m_colorLocation = glGetAttribColor(m_shader->getProgram(), "color");
glEnableVertexAttribArray(m_colorLocation);

// Bind the vertex buffer object to the input assembler.
glBindBuffer(GL_ARRAY_BUFFER, m_geometryBuffer);
glVertexAttribPointer(m_positionLocation, 4, GL_FLOAT, GL_FALSE, 0, NULL);
glBindBuffer(GL_ARRAY_BUFFER, m_colorBuffer);
glVertexAttribPointer(m_colorLocation, 3, GL_FLOAT, GL_FALSE, 0, NULL);

// Draw a triangle with 3 vertices.
glDrawArray(GL_TRIANGLES, 0, 3);
```

Direct3D rendering code

C++

```
// Bind the vertex shader and pixel shader to the pipeline.
m_d3dDeviceContext->VSSetShader(vertexShader.Get(),nullptr,0);
m_d3dDeviceContext->PSSetShader(pixelShader.Get(),nullptr,0);

// Declare the inputs that the shaders expect.
m_d3dDeviceContext->IASetInputLayout(inputLayout.Get());
m_d3dDeviceContext->IASetVertexBuffers(0, 1, vertexBuffer.GetAddressOf(),
&stride, &offset);

// Set the primitive's topology.
m_d3dDeviceContext-
>IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);

// Draw a triangle with 3 vertices. triangleVertices is an array of 3
vertices.
m_d3dDeviceContext->Draw(ARRAYSIZE(triangleVertices),0);
```

Related topics

- Port from OpenGL ES 2.0 to Direct3D 11

Feedback

Frequently asked questions

FAQ

Things not working the way you expected? Look through this page of frequently asked questions. Also check out the Known issues topic and the Developing Universal Windows apps forum.

Why aren't my games and apps working?

If your games and apps aren't working, or if you don't have access to the store or to Live services, you are probably running in Developer Mode. To figure out which mode you're currently in, press the **Home** button on your controller. If this takes you to Dev Home instead of the retail Home experience, you're in Developer Mode. If you want to play games, you can open Dev Home and switch back to Retail Mode by using the **Leave developer mode** button.

Why can't I connect to my Xbox One using Visual Studio?

Start by verifying that you are running in Developer Mode, and not in Retail Mode. You cannot connect to your Xbox One when it is in Retail Mode. To figure out which mode you're currently in, press the **Home** button on your controller. If you see Gold/Live content instead of Dev Home, you're in Retail Mode and you need to run the Dev Mode Activation app to switch to Developer Mode.

> **Note**
>
> You must have a user signed in to deploy an app.

For more information, see Fixing deployment failures later on this page.

How do I switch between Retail Mode and Developer Mode?

Follow the Xbox One Developer Mode Activation instructions to understand more about these states.

How do I know if I am in Retail Mode or Developer Mode?

Follow the Xbox One Developer Mode Activation instructions to understand more about these states.

To figure out which mode you're currently in, press the **Home** button on your controller.

- If this takes you to Dev Home, you're in Developer Mode.
- If you see Gold/Live content, you're in Retail Mode.

Will my games and apps still work if I activate Developer Mode?

Yes, you can switch from Developer Mode to Retail Mode, where you can play your games. For more information, see the Xbox One Developer Mode Activation page.

Can I develop and publish x86 apps for Xbox?

Xbox no longer supports x86 app development or x86 app submissions to the store.

Will I lose my games and apps or saved changes?

If you decide to leave the Developer Program, you won't lose your installed games and apps. In addition, as long as you were online when you played them, your saved games are all saved on your Live account cloud profile, so you won't lose them.

How do I leave the Developer Program?

See the Xbox One Developer Mode Deactivation topic for details about how to leave the Developer Program.

I sold my Xbox One and left it in Developer Mode. How do I deactivate Developer Mode?

If you no longer have access to your Xbox One, you can deactivate it in Windows Partner Center. For details, see the **Deactivate your console using Partner Center** section in the Xbox One Developer Mode Deactivation topic.

I left the Developer Program using Partner Center but I'm in still Developer Mode. What do I do?

Start Dev Home and select the **Leave developer mode** button. This will restart your console in Retail Mode.

Can I publish my app?

You can publish apps through Partner Center if you have a developer account. UWP apps created and tested on a retail Xbox One console will go through the same ingestion, review, and publication process that Windows conducts today, with additional reviews to meet today's Xbox One standards.

Can I publish my game?

You can use UWP and your Xbox One in Developer Mode to build and test your games on Xbox One. To publish UWP games, you must register with ID@XBOX or be part of the Xbox Live Creators Program. For more info, see Developer Program Overview.

Will the standard Game engines work?

Check out the Known issues page for this release.

What capabilities and system resources are available to UWP games on Xbox

One?

For information, see System resources for UWP apps and games on Xbox One.

If I create a DirectX 12 UWP game, will it run on my Xbox One in Developer Mode?

For information, see System resources for UWP apps and games on Xbox One.

Will the entire UWP API surface be available on Xbox?

Check out the Known issues page for this release.

Fixing deployment failures

If you can't deploy your app from Visual Studio, these steps may help you fix the problem. If you get stuck, ask for help on the forum.

> **Note**
>
> You must have a user signed in to deploy an app. If you receive a 0x87e10008 error message, make sure you have a user signed in and try again.

If Visual Studio cannot connect to your Xbox One:

1. Make sure that you are in Developer Mode (discussed earlier on this page).
2. Make sure that you have set up your development PC correctly. Did you follow *all* of the directions in Getting started with UWP app development on Xbox One?
3. If you haven't yet, read through the Development environment setup topic and the Introduction to Xbox One tools topic.
4. Make sure that you can “ping” your console IP address from your development PC.

> **Note**

In order to get the best deployment performance, we recommend that you use a wired connection to your console.

5. Make sure that you are using the Universal (Unencrypted Protocol) in the Authentication drop-down list on the **Debug** tab. For more details, see Development environment setup.

If I'm building an app using HTML/JavaScript, how do I enable Gamepad navigation?

TVHelpers is a set of JavaScript and XAML/C# samples and libraries to help you build great Xbox One and TV experiences in JavaScript and C#. TVJS is a library that helps you build premium UWP apps for Xbox One. TVJS includes support for automatic controller navigation, rich media playback, search, and more. You can use TVJS with your hosted web app just as easily as with a packaged web UWP app with full access to the Windows Runtime APIs.

For more information, see the TVHelpers project and the project wiki.

See also

- Known issues with UWP on Xbox One
- UWP on Xbox One

Feedback

Xbox services overview

Article • 03/28/2024

Xbox services are a collection of micro-services that expose features such as profile, friends and presence, stats, leaderboards, achievements, multiplayer, and matchmaking.

Using Xbox services, you can also add Xbox services to Windows PC and Xbox console titles.

Xbox services

Xbox services are a premier gaming network that connects millions of gamers across the world.

Here are some reasons to add Xbox services to your game.

- Xbox services unite gamers across Xbox One (or later) and Windows 10, so gamers can play with their friends and connect with a massive community of players.
- Xbox services let players build a gaming legacy by unlocking achievements, sharing epic game clips, amassing gamerscore, and perfecting their avatar.
- Xbox services let gamers play and pick up where they left off on another Xbox console or PC, bringing all their saves from another device.
- With over 1 billion multiplayer matches played each month, Xbox services are built for performance, speed, and reliability.
- With cross-device multiplayer, gamers can play with your friends regardless of whether they play on Xbox One (or later) or Windows 10 PC.

How Xbox services work

Xbox services data is stored in the cloud. It can be accessed by using REST endpoints and secure web sockets that are accessible from a set of client-side APIs designed for game developers.

We recommend the use of client-side APIs, known as Xbox Services API (XSAPI), that wrap the REST functionality for development. For more information, see Xbox Services API overview.

Xbox services feature areas

Player Data

Player Data drives the usage of player stats, achievements, and leaderboards.

Expand table

Feature	Description
Player Data	A brief overview of the Player Data system, as well as guidance on how to best incorporate stats, leaderboards, and achievements into your title.
Stats and Leaderboards	Stats is the foundation of leaderboards. When they're intelligently incorporated, they help bring out your users' competitiveness.
Achievements	Achievements is one of the most well-known features for players signed in to Xbox services. It's a great driver of player engagement. Learn how to use them in your title.

Social features

Social features can organically grow your audience, spreading awareness to over 55 million active users.

Expand table

Feature	Description
Social	If you can sign a user into the Xbox network (also known as Xbox Live), you can start using Xbox services' social features. This includes utilizing a user's social graph, Rich Presence, Friends list (People system), activity feed (presence strings), and reputation.

Multiplayer features

Multiplayer is a great way to extend the lifetime of your title and keep gameplay experiences fresh.

Xbox services provides extensive multiplayer and matchmaking features.

You also have several options of API that provide varying levels of simplicity versus flexibility.

Expand table

Feature	Description
Multiplayer overview	If you are new to Xbox services multiplayer development, or are unfamiliar with new APIs such as Multiplayer Manager, start here.
Common multiplayer scenarios	Suggestions and guidance on how you might incorporate multiplayer into your title.
Multiplayer Manager (MPM)	Multiplayer Manager provides a high-level API focused on common multiplayer scenarios, including adding multiplayer functionality by managing sessions, matchmaking, and game invites. Provides a state- and event-based programming model.

Cloud Storage

Cloud Storage provides both **Title Storage** and **Connected Storage**.

These are two different but complementary services.

- Connected Storage allows you to implement game saves in the cloud, which will roam across devices, regardless of where a user is signed in.
- Title Storage lets you store blobs of data that can be per user or per title and shared across different users.

Expand table

Feature	Description
Connected Storage vs. Title Storage	Xbox services Cloud Storage includes Connected Storage to store game state, and Title Storage to store player statistics and assets.
Connected Storage	Connected Storage saves and loads gameplay data and other state data across devices. Title data is stored locally and also synced to the cloud.

See also

- Getting started with Xbox services
- Xbox Services API overview

What's New for developers in Windows 10 build 19041

Article • 10/20/2022

This is a collection of articles providing information and guidance on features added in Windows 10 build 19041 (also known as Version 2004). For a full list of new namespaces added to the Windows SDK, see the Windows 10 build 19041 API changes. For more information on the highlighted features of Windows 10, see What's cool in Windows 10.

Windows 10 apps

Expand table

Feature	Description
Bluetooth audio playback	Enable audio playback from remote Bluetooth-connected devices shows you how to use AudioPlaybackConnection to enable Bluetooth-connected remote devices to play back audio on the local machine, enabling scenarios such as configuring a PC to behave like a Bluetooth speaker and allowing users to hear audio from their phone.
C# app porting	We've documented the process of porting a C# application to C++/WinRT. Porting the Clipboard sample to C++/WinRT from C# is contextual, and based on a particular real-world porting experience. Its companion topic Move to C++/WinRT from C# is a more encyclopedic look at the technical details and steps involved in porting.
C++/WinRT	Read about the updates to C++/WinRT regarding build-time and run-time performance improvements (achieved in concert with the Visual C++ compiler team), in Rollup of recent improvements/additions. For C++/WinRT, we added more info to these topics: porting from C++/CX, porting from C#, Simple C++/WinRT Windows UI Library example, Concurrency, get_unknown(), and XAML custom (templated) controls with C++/WinRT.
DirectX	We brought several DirectX-related "What's new" topics up to date for several past releases of Windows, from the Creators Update to Windows 10, version 1903. What's new in DirectWrite, DXGI 1.6 improvements, and What's new in Direct3D 12.
DirectXMath	We published 21 new DirectXMath topics, covering two matrix structures and their member functions and free functions. The XMFLOAT3X4 structure is an example.
Direct3D	Using DirectX with high dynamic range displays and advanced color provides a list of best practices for Windows high-dynamic-rnge apps.

Feature	Description
	A new ID3D11On12Device2 interface, and its methods, enable you to take resources created through the Direct3D 11 APIs and use them in Direct3D 12.
Direct3D 12	The Direct3D 12 Core 1.0 Feature Level has been added, for use by *compute-only* devices. New topics habe been added for the ID3D12Debug3 interface.
Direct ML	There 18 operators have been added to DirectML, the low-level hardware-accelerated API on which WinML is built. An example is the DML_ACTIVATION_SHRINK_OPERATOR_DESC structure.
Error reporting	The RoFailFastWithErrorContextInternal2 function has been added to Win32, which raises an exception which can contain additional error context.
Machine Learning	Windows Machine Learning now supports ONNX version 1.4 and opset 9. The CloseModelOnSessionCreation API allows you to save memory by closing a learning model automatically once it is no longer needed.
Wi-Fi	Several new Native WiFi functions and structures have been added, such as the WlanDeviceServiceCommand function.
Wi-Fi Hotspot 2	Provision a Wi-Fi profile via a website describes new functionality for Wi-Fi Hotspot 2.
Windows Holographic interop	The windows.graphics.holographic.interop.h header has been added, with 17 Win32 APIs. The APIs are for interoperating between Win32 and Windows Runtime. While the APIs were added in Windows 10 build 18362, the header is new for build 19041.
Windows Sockets	Enhancements have been made to the Windows Sockets 2 SPI content. An example of one of the many topics we improved and augmented is the LPWSPEVENTSELECT callback function topic.
XAML Islands - basics	Host UWP XAMl controls in your desktop Windows apps with XAML islands. Learn how to Use XAML Islands to host a UWP XAML control in a C# WPF app, and host a standard UWP control in a C++ Win32 app.
XAML Islands - custom controls	The Microsoft.Toolkit.Win32.UI.XamlApplication and Microsoft.Toolkit.Win32.UI.SDK NuGet packages make it easier to host custom UWP XAML controls in .NET and C++ Win32 apps. For step-by-step walkthroughs, see Host a custom UWP control in a WPF app and Host a custom UWP control in a C++ Win32 app. Finally, for guidance on more complicated C++ Win32 scenarios, see Advanced scenarios for XAML Islands.

Build with Windows

Expand table

Feature	Description
Windows development environment	The Windows development environment docs provide resources for using Windows to develop across a variety of platforms, to accomplish whatever development goals you might have.
Python on Windows	The Python on Windows section provides information for developers new to the Python language, as well as devs looking to optimize their Python development with other tools available on Windows. Learn how to set up your Python environment for web development and database interaction.
NodeJS on Windows	The recommended setup for your Node.js development environment provides detailed guidelines for advanced developers deploying to Linux servers. Also available are setup instructions for popular Node.js web frameworks, database interaction, and Docker containers.
Mac to Windows	Our guide to changing your dev environment is geared towards users transitioning their development platform from Mac to Windows, and provides mappings for comparable shortcuts and development utilities.
Windows Terminal	A modern terminal application for users of command line tools and shells like Command Prompt, PowerShell, and Windows Subsystem for Linux (WSL). Its main features include multiple tabs, panes, Unicode and UTF-8 character support, a GPU accelerated text rendering engine, and the ability to create your own themes and customize text, colors, backgrounds, and shortcut key bindings.
WSL 2	A new version of the Windows Subsystem for Linux (WSL) is now available. WSL 2 features reconfigured architecture to run an actual Linux kernel on Windows, increasing file system performance and adding full system call compatibility. This new architecture changes how Linux binaries interact with Windows and your computer's hardware, but still provides the same user experience as in the previous version of WSL. Each individual Linux distribution can run as a WSL1 or WSL2 distro, can be run side by side, and can be changed at any time. Install WSL 2 to get started. Explore further information on changes between WSL 1 and WSL 2. Check out the Frequently Asked Questions about WSL 2.

MSIX, packaging, and deployment

Expand table

Feature	Description
MSIX	Significant updates to the MSIX packaging format have been made since the last release of the Windows 10 SDK.

Feature	Description
Packaging with services	MSIX and the MSIX Packaging Tool now support app packages that contain services.
Scripts in MSIX packages	You can use the Package Support Framework (PSF) to run scripts in an MSIX app package, enabling IT Pros to customize an application dynamically to the user's environment after it is packaged using MSIX.
Enforced package integrity	You can now enforce package integrity on the contents of MSIX packages by using the uap10:PackageIntegrity element in your package manifest. You can also enforce package integrity when you create MSIX packages via the MSIX Packaging Tool.
Package with external location	You can grant package identity by building and registering a package with external location (see Grant package identity by packaging with external location). This option is useful if you're unable to adopt MSIX for installing your desktop app, and still use Windows extensibility features that require package identity.
Hosted apps	You can now create hosted apps. Hosted apps share the same executable and definition as a parent host app, but they look and behave like a separate app on the system. Hosted apps are useful for scenarios where you want a component (such as an executable file or a script file) to behave like a standalone Windows app, but the component requires a host process in order to execute. A hosted app can have its own start tile, identity, and deep integration with Windows features such as background tasks, notifications, tiles, and share targets.

Windows UI Library (WinUI)

Expand table

Feature	Description
WinUI 2.4	WinUI 2.4 is the latest public release of the Windows UI Library. All versions of WinUI provide a wide assortment of official UI controls for your Windows apps, and are supplied as a NuGet package independent of the Windows SDK, so they work on earlier versions of Windows 10. Follow these instructions to install WinUI.
RadialGradientBrush	New in WinUI 2.4, a RadialGradientBrush is drawn within an ellipse defined by Center, RadiusX, and RadiusY properties. Colors for the gradient start at the center of the ellipse and end at the radius.
ProgressRing	New in WinUI 2.4, the ProgressRing control is used for modal interactions where the user is blocked until the ProgressRing disappears. Use this control if an operation requires that most interaction with the app be suspended until the operation is complete.

Feature	Description
TabView	Updates to the TabView control provide you with more control over how to render tabs. You can set the width of unselected tabs and show just an icon to save screen space, and can also hide the close button on unselected tabs until the user hovers over the tab.
TextBox controls	When dark theme is enabled, the background color of TextBox family controls now remains dark by default on text insertion. Affected controls are TextBox, RichEditBox, PasswordBox, Editable ComboBox, and AutoSuggestBox.
NavigationView	The NavigationView control now supports hierarchical navigation and includes Left, Top, and LeftCompact display modes. A hierarchical NavigationView is useful for displaying categories of pages, identifying pages with related child-pages, or using within apps that have hub-style pages linking to many other pages.
Windows UI Gallery	Examples of each WinUI feature are available in the XAML Controls Gallery. Download it on the Microsoft Store, or view the source code on Github.
Previous versions	Since the previous major release of the Windows 10 SDK, WinUI 2.3 and WinUI 2.2 were also released, providing further new UI features for Windows devs.

Samples

The following sample apps have been updated to target Windows 10 build 19041.

- Remote Sessions (Quiz Game)
- Customer Orders Database
- RSS Reader
- Marble Maze
- Photo Editor
- Lunch Scheduler
- Coloring Book
- Hue Light Controller
- Photo Lab
- Family Notes

Videos

Windows Terminal: the secret to command line happiness!

Learn about how to customize the Windows Terminal for your workflow, and see demos of its features in action. Check out the video, then read the docs for more information.

WSL2: Code faster on the Windows Subsystem for Linux

Learn all about WSL2, the new version of the Windows Subsystem for Linux, and what changes have been made to improve performance. Check out the video, then read the docs for more information.

MSIX: Package desktop apps for Windows 10. Replace outdated installers.

Learn about MSIX, the package format for installing Windows apps, including how to package your existing code with Visual Studio and how to deploy and distribute your app. Check out the video, then read the docs for more information.

Feedback

New APIs in Windows 10 build 19041

Article • 10/20/2022

New and updated API namespaces have been made available to developers in Windows 10 build 19041 (Also known as SDK version 2004). Below is a full list of documentation published for namespaces added or modified in this release.

For information on APIs added in the previous public release, see New APIs in the Windows 10 October Update.

Windows.AI

Windows.AI.MachineLearning

LearningModelSessionOptions

LearningModelSessionOptions.CloseModelOnSessionCreation

Windows.ApplicationModel

Windows.ApplicationModel.Background

BackgroundTaskBuilder

BackgroundTaskBuilder.SetTaskEntryPointClsid

BluetoothLEAdvertisementPublisherTrigger

BluetoothLEAdvertisementPublisherTrigger.IncludeTransmitPowerLevel
BluetoothLEAdvertisementPublisherTrigger.IsAnonymous
BluetoothLEAdvertisementPublisherTrigger.PreferredTransmitPowerLevelInDBm
BluetoothLEAdvertisementPublisherTrigger.UseExtendedFormat

BluetoothLEAdvertisementWatcherTrigger

BluetoothLEAdvertisementWatcherTrigger.AllowExtendedAdvertisements

Windows.ApplicationModel.ConversationalAgent

ActivationSignalDetectionConfiguration

ActivationSignalDetectionConfiguration
ActivationSignalDetectionConfiguration.ApplyTrainingData
ActivationSignalDetectionConfiguration.ApplyTrainingDataAsync
ActivationSignalDetectionConfiguration.AvailabilityChanged
ActivationSignalDetectionConfiguration.AvailabilityInfo
ActivationSignalDetectionConfiguration.ClearModelData
ActivationSignalDetectionConfiguration.ClearModelDataAsync
ActivationSignalDetectionConfiguration.ClearTrainingData
ActivationSignalDetectionConfiguration.ClearTrainingDataAsync
ActivationSignalDetectionConfiguration.DisplayName
ActivationSignalDetectionConfiguration.GetModelData
ActivationSignalDetectionConfiguration.GetModelDataAsync
ActivationSignalDetectionConfiguration.GetModelDataType
ActivationSignalDetectionConfiguration.GetModelDataTypeAsync
ActivationSignalDetectionConfiguration.IsActive
ActivationSignalDetectionConfiguration.ModelId
ActivationSignalDetectionConfiguration.SetEnabled
ActivationSignalDetectionConfiguration.SetEnabledAsync
ActivationSignalDetectionConfiguration.SetModelData
ActivationSignalDetectionConfiguration.SetModelDataAsync
ActivationSignalDetectionConfiguration.SignalId
ActivationSignalDetectionConfiguration.TrainingDataFormat
ActivationSignalDetectionConfiguration.TrainingStepsCompleted
ActivationSignalDetectionConfiguration.TrainingStepsRemaining

ActivationSignalDetectionTrainingDataFormat

ActivationSignalDetectionTrainingDataFormat

ActivationSignalDetector

ActivationSignalDetector
ActivationSignalDetector.CanCreateConfigurations
ActivationSignalDetector.CreateConfiguration
ActivationSignalDetector.CreateConfigurationAsync
ActivationSignalDetector.GetConfiguration

ActivationSignalDetector.GetConfigurationAsync
ActivationSignalDetector.GetConfigurations
ActivationSignalDetector.GetConfigurationsAsync
ActivationSignalDetector.GetSupportedModelIdsForSignalId
ActivationSignalDetector.GetSupportedModelIdsForSignalIdAsync
ActivationSignalDetector.Kind
ActivationSignalDetector.ProviderId
ActivationSignalDetector.RemoveConfiguration
ActivationSignalDetector.RemoveConfigurationAsync
ActivationSignalDetector.SupportedModelDataTypes
ActivationSignalDetector.SupportedPowerStates
ActivationSignalDetector.SupportedTrainingDataFormats

ActivationSignalDetectorKind

ActivationSignalDetectorKind

ActivationSignalDetectorPowerState

ActivationSignalDetectorPowerState

ConversationalAgentDetectorManager

ConversationalAgentDetectorManager
ConversationalAgentDetectorManager.Default
ConversationalAgentDetectorManager.GetActivationSignalDetectors
ConversationalAgentDetectorManager.GetActivationSignalDetectorsAsync
ConversationalAgentDetectorManager.GetAllActivationSignalDetectors
ConversationalAgentDetectorManager.GetAllActivationSignalDetectorsAsync

DetectionConfigurationAvailabilityChangeKind

DetectionConfigurationAvailabilityChangeKind
DetectionConfigurationAvailabilityChangedEventArgs

DetectionConfigurationAvailabilityChangedEventArgs

DetectionConfigurationAvailabilityChangedEventArgs.Kind

DetectionConfigurationAvailabilityInfo

DetectionConfigurationAvailabilityInfo
DetectionConfigurationAvailabilityInfo.HasLockScreenPermission
DetectionConfigurationAvailabilityInfo.HasPermission
DetectionConfigurationAvailabilityInfo.HasSystemResourceAccess
DetectionConfigurationAvailabilityInfo.IsEnabled

DetectionConfigurationTrainingStatus

DetectionConfigurationTrainingStatus

Windows.ApplicationModel

AppInfo

AppInfo.Current
AppInfo.GetFromAppUserModelId
AppInfo.GetFromAppUserModelIdForUser
AppInfo.Package

IAppInfoStatics

IAppInfoStatics
IAppInfoStatics.Current
IAppInfoStatics.GetFromAppUserModelId
IAppInfoStatics.GetFromAppUserModelIdForUser

Package

Package.EffectiveExternalLocation
Package.EffectiveExternalPath
Package.EffectivePath
Package.GetAppListEntries
Package.GetLogoAsRandomAccessStreamReference
Package.InstalledPath
Package.IsStub
Package.MachineExternalLocation
Package.MachineExternalPath
Package.MutablePath
Package.UserExternalLocation
Package.UserExternalPath